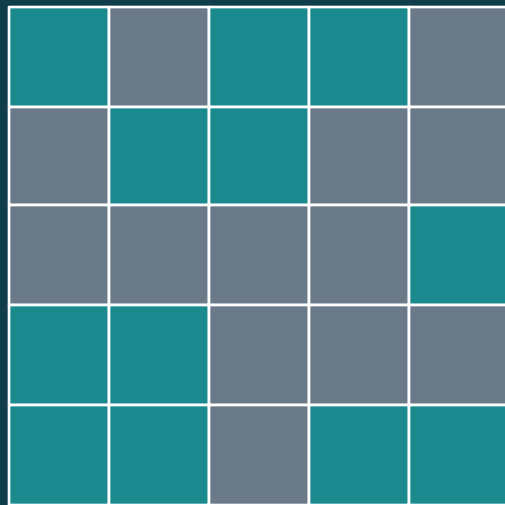


Contagem paralela de objetos em uma matriz binária

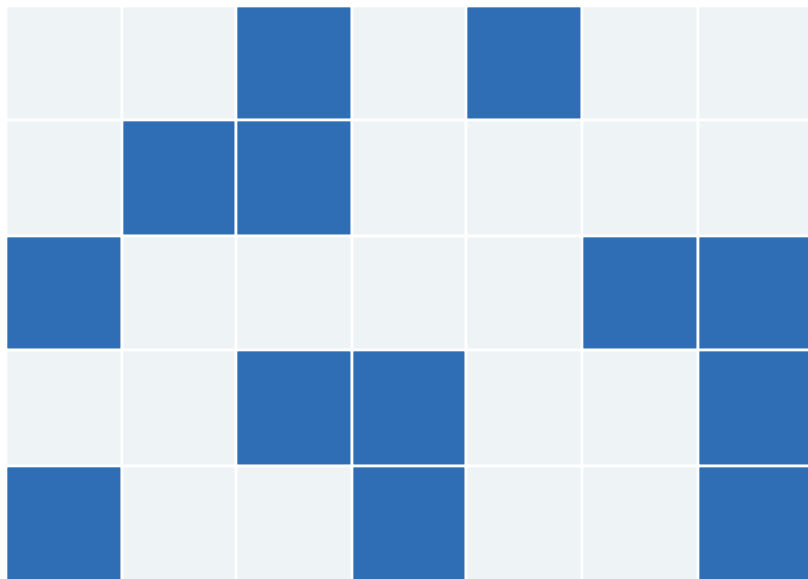
Sequencial x Pthreads, conectividade 8

Sistemas Operacionais 2026/II | PUCRS

Isadora Santos, Jhone Salvador e Nathan Schmitt



0 problema: contar objetos na matriz



Exemplo 5x7: 5 objetos

0 = fundo, 1 = primeiro plano

Objeto: região de 1s ligados por aresta ou por canto

Conectividade 8: os 8 vizinhos contam, inclusive a diagonal

Entregas: versão sequencial (referência) e versão paralela com Pthreads, mesmo resultado, ANSI C (C89)

Arquitetura do código

`matriz.c / .h`

Leitura do arquivo (LxC + valores) e liberação da matriz

`floodfill.c / .h`

Contagem sequencial: flood fill iterativo + matriz visitado

`workers.c / .h`

Contagem paralela: threads, rotulação local, union-find

`main.c`

Programa principal: argumentos, cronômetro, roda as duas versões

Fluxo: lerMatriz > dividirTrabalho(p) > versaoSequencial() > imprime
objetos e tempo > liberarMatriz

Versão sequencial: flood fill iterativo

```
para cada (i,j):  
    se M[i][j]==1 e !vis[i][j]:  
        cont++ ; vis[i][j]=1  
        push(i,j)  
        enquanto pilha nao vazia:  
            (r,c) = pop  
            para dr,dc em {-1,0,1},  
                exceto (0,0):  
                se dentro, M==1, !vis:  
                    vis=1 ; push  
retorne cont
```

1

Matriz visitado

a entrada não é alterada: sequencial e paralelo usam os mesmos dados

2

Pilha explícita

sem recursão, sem estouro de pilha em matrizes grandes

3

Marca ao empilhar

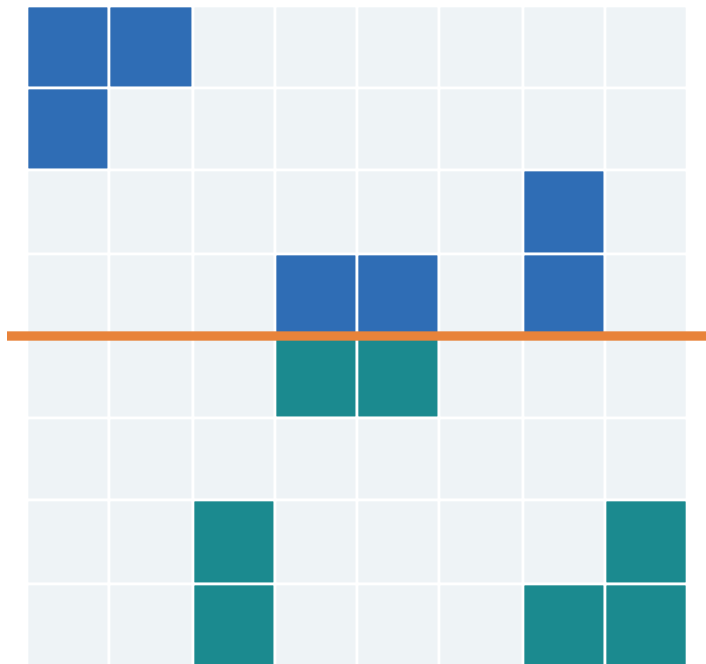
cada célula entra uma vez: pilha de no máximo $L \cdot C$

4

Custo

tempo e memória $O(L \cdot C)$

0 desafio: um objeto cruza as faixas



Exemplo 3 (8x8), $p = 2$: faixa 0 = linhas 0-3, faixa 1 = linhas 4-7

Contagens locais

faixa 0: 3 objetos

faixa 1: 3 objetos

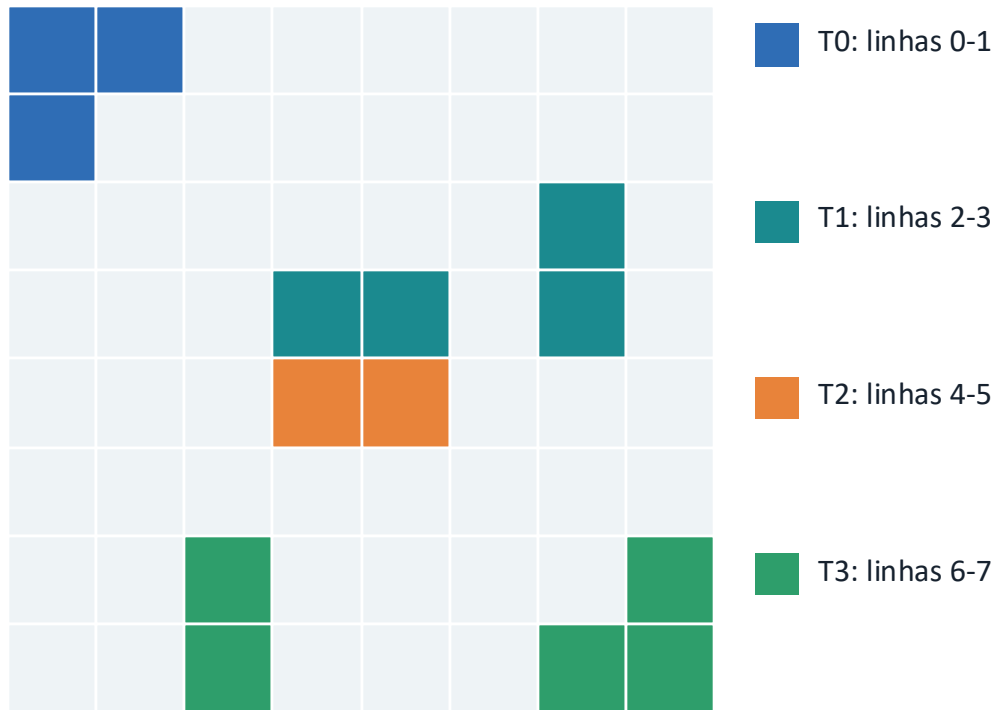
Soma = 6 (errado)

Resposta correta = 5

O objeto central está nas linhas 3 e 4 e foi contado duas vezes

Somar contagens locais não basta: é preciso reconhecer e unificar componentes equivalentes nas fronteiras.

Decomposição: faixas de linhas, uma por thread



Como as faixas são calculadas

$\text{faixa} = L / p$, e as primeiras $L \bmod p$ faixas recebem 1 linha a mais

Ex.: $L=10$, $p=4$ resulta em 3, 3, 2, 2

Se $p > L$, p vira L (sem faixas vazias)

p vem da linha de comando:
`./contador arquivo p`

Fase paralela e sincronização

Dado	Quem escreve	Proteção
dados (entrada)	ninguém (só leitura)	não precisa
labels (L x C)	cada thread, só nas suas linhas	partição disjunta
pilha do flood fill	própria thread	uma por thread
Worker[t]	principal, antes do create	lido só pela thread t
union-find	principal, após o join	pthread_join

Sem mutex

não há escrita compartilhada, logo
não há corrida

Sem deadlock

não há locks para esperar

join = barreira

consolidação só começa com todas
as threads concluídas

Consolidação: fronteiras e union-find

1

Rótulo local

linha · C + coluna + 1 da célula-semente: único em toda a matriz

2

Fronteira

última linha da faixa com a primeira da seguinte: colunas j-1, j e j+1 (pega a diagonal)

3

União

unir(rótulo A, rótulo B) com path halving

4

Contagem

soma das contagens locais – uniões efetivas

Exemplo 3, p = 2

semente (3, 3): $3 \cdot 8 + 3 + 1 = 28$

semente (4, 3): $4 \cdot 8 + 3 + 1 = 36$

fronteira linhas 3/4, col 3-4

unir(28, 36)

6 componentes locais

6 – 1 união = 5 objetos

O que é paralelo e o que continua sequencial

Etapa	Execução	Custo
Leitura do arquivo (fora do tempo medido)	sequencial	$O(L \cdot C)$
Particionamento em faixas	sequencial	$O(p)$
Rotulação local (flood fill)	PARALELA	$O(L \cdot C / p)$
Uniões nas fronteiras	sequencial	$O(p \cdot C)$
Alocar rótulos + iniciar union-find	sequencial	$O(L \cdot C)$

Limite de aceleração (Amdahl): a alocação $O(L \cdot C)$ na thread principal e a criação das threads limitam $S(p)$. Matrizes pequenas não compensam esse custo, por isso medimos com uma matriz grande.

Testes de correção

Matriz	Dim.	Esperado	Seq.	Paralelo (1, 2, 4, 8 threads)
Ex. 1	5x5	3	3	3 em todas (aprovado)
Ex. 2	6x8	4	4	4 em todas (aprovado)
Ex. 3	8x8	5	5	5 em todas (aprovado)
Ex. 4	9x12	6	6	6 em todas (aprovado)
Ex. 5	12x12	7	7	7 em todas (aprovado)

5

casos adicionais

zeros, coluna multifaixa, diagonal,
uma linha, tudo 1

300

matrizes aleatórias

seq x par, 1 a 64 threads, 0
divergências

0

aviso de compilação

cc -std=c89 -Wall -Wextra -pedantic

Desempenho: metodologia e resultados

Metodologia

- Mesma matriz grande nas duas versões
- Só a contagem é medida (sem ler arquivo)
- `clock_gettime(CLOCK_MONOTONIC)`
- 5 repetições por configuração, mediana
- $S(p) = T_{seq} / T_{par}(p)$
- $E(p) = S(p) / p$
- Resultado conferido a cada repetição

Versão	p	Mediana (ms)	S(p)	E(p)
Sequencial	1	121,83	1,00	1,00
Paralela	1	148,39	0,82	0,82
Paralela	2	83,29	1,46	0,73
Paralela	4	53,72	2,27	0,57
Paralela	8	40,57	3,00	0,38

Matriz 2000x2000 (~50% de 1s, semente 2026, 13345 objetos). Ryzen 7 7735HS, 8 núcleos/16 threads, Linux Mint 22.3, GCC 13.3 -O2. Dados: results/medicoes.csv

Problemas encontrados e corrigidos

1

Recursão no flood fill > **pilha explícita**

800x800 com 1 thread estourava a pilha

2

Threads invadiam a faixa vizinha > **flood fill limitado à faixa**

eliminou a condição de corrida

3

Objeto na fronteira contado 2x > **union-find nas bordas**

total = soma local – uniões efetivas

4

Erros de pthread e malloc sem tratamento > **retorna -1 e libera os recursos**

falha distinguível de resultado

Conclusões

Entregue

- Sequencial e paralela com o mesmo resultado
- Fronteiras (inclusive diagonais) unificadas
- Sem corrida, sem deadlock
- Testes obrigatórios e aleatórios aprovados

Limitações

- Faixas estáticas de linhas
- Uniões e varreduras só na thread principal
- Matrizes pequenas não compensam threads

Trabalho futuro

- Fila dinâmica de blocos
- Consolidação paralela
- Versão com processos POSIX